1155194577

AIST3030/ESTR3114: Homework 3

Deadline: November 12, 11:00 pm on Gradescope

In this homework, we will implement the **coordinate descent** method for solving optimization problems. Here we consider the following unconstrained optimization problem:

$$\min_{\mathbf{x}\in\mathbb{R}^d} \quad f(\mathbf{x})$$

The coordinate descent method is an iterative algorithm, which at every iteration t fixes all coordinates of $\mathbf{x} = [x_1, x_2, \ldots, x_d] \in \mathbb{R}^d$ except the coordinate $x_{t \bmod d}$ ($t \bmod d$ denotes the remainder[1] of t divided by d) and finds the optimal value of scalar variable $x_{t \bmod d}$. Therefore, the update rule for coordinate descent at iteration t finds $\mathbf{x}^{(t+1)}$ from $\mathbf{x}^{(t)}$ as follows

$$\forall i \in \{1, 2, \ldots, d\}: \quad x_i^{(t+1)} = \begin{cases} \operatorname*{argmin}_{y\in\mathbb{R}} f\Big(\big[x_1, \ldots, x_{i-1}, y, x_{i+1}, \ldots, x_d\big]\Big) & \text{if } \ i = t \bmod d \\ x_i^{(t)} & \text{otherwise.} \end{cases}$$

The coordinate decent algorithm is especially helpful in cases that we can **analytically and efficiently** solve the problem for every coordinate at a time, i.e. scalar optimization is computationally cheaper than multivariate optimization.

Suppose that we have the Ridge-Regression objective function for matrix $A \in \mathbb{R}^{m\times d}$ and vector $\mathbf{b} \in \mathbb{R}^m$ and coefficient $\lambda > 0$:

$$\min_{\mathbf{x}\in\mathbb{R}^d} \quad f(\mathbf{x}) := \big\|A\mathbf{x} - \mathbf{b}\big\|_2^2 + \lambda\big\|\mathbf{x}\big\|_2^2$$

1. Use the first-order necessary condition to analytically characterize the optimal solution and find the closed-form version of the argmin in the coordinate descent update rule.

   Hint: Note that for every index $i \in \{1, \ldots, d\}$, $f(\mathbf{x})$ can be written as

   $$f(\mathbf{x}) \ = \ \big\|x_i\mathbf{a}_i - \big(\mathbf{b} - A_{-i}\mathbf{x}_{-i}\big)\big\|_2^2 + \lambda x_i^2 + \lambda\|\mathbf{x}_{-i}\|_2^2$$

   where $\mathbf{a}_i$ is the ith column of matrix A, A_{-i} is the matrix A **without** its ith column, and $\mathbf{x}_{-i}$ is the vector $\mathbf{x}$ **without** its ith entry x_i. Then, what is the optimal x_i for a fixed $\mathbf{x}_{-i}$?

In the following questions, consider $A \in \mathbb{R}^{m\times d}$ and $\mathbf{b} \in \mathbb{R}^m$ where we pick $d = 100$ and $m = 10$.

2. Write the **pseudocode** for the coordinate descent algorithm to solve the Ridge Regression problem. The pseudocode should describe the steps of the iterative coordinate descent algorithm.

[1] Note that we set $d \bmod d = d$ instead of the commonly considered zero remainder.

3. Use the numpy package to generate independent normally-distributed entries according to $\mathcal{N}(0,1)$ for the entries of A and $\mathbf{b}$

```
import numpy as np
A = #Add your answer here
b = #Add your answer here
```

4. Write a python function for the scalar optimization of coordinate descent. The function should take A, $\mathbf{b}$ as well as the current optimization variable $\mathbf{x}$ and the to-be-optimized index t as input, and output the next vector $\mathbf{x}$ with the optimized variable at index t

```
def coordinate-opt(A , b , lambda , x , t ):
    # Add your response
    x_updated = # Add your response
    return x_updated
```

5. Now, write a code for applying coordinate descent to the Ridge Regression problem over $T = 2000$ iterations for regularization coefficients $\lambda = 0.1, 1.0, 10.0, 100.0$ initialized at $\mathbf{x}_{\text{init}}$. Plot the optimization objective value over the $T = 2000$ iterations

```
import matplotlib.pyplot as plt

def Ridge_Regression_solver(A, b , lambda , x_init , T=2000):
    # add your response here
    ans = # add your response here
    return ans
```

$\mathbf{x} \in \mathbb{R}^d$ [illegible] $\|\cdot\|_2$ [illegible] $\|\cdot\|_2$

Problem 1

1. Use the first-order necessary condition to analytically characterize the optimal solution and find the closed-form version of the argmin in the coordinate descent update rule.

   Hint: Note that for every index $i \in \{1, \ldots, d\}$, $f(\mathbf{x})$ can be written as

   $$f(\mathbf{x}) = \|x_i \mathbf{a}_i - (\mathbf{b} - A_{-i}\mathbf{x}_{-i})\|_2^2 + \lambda x_i^2 + \lambda \|\mathbf{x}_{-i}\|_2^2$$

   where $\mathbf{a}_i$ is the ith column of matrix A, A_{-i} is the matrix A **without** its ith column, and $\mathbf{x}_{-i}$ is the vector $\mathbf{x}$ **without** its ith entry x_i. Then, what is the optimal x_i for a fixed $\mathbf{x}_{-i}$?

scalar : x_i

$$f(x) = [x_i a_i - (\vec{b} - A_{-i} x_{-i})]^T [x_i a_i - (b - A_{-i} x_{-i})] + \lambda x_i^2 + \lambda x_{-i}^T x_{-i}$$

$$f(x) = [(x_i a_i)^T - (\vec{b} - A_{-i} x_{-i})^T][x_i a_i - (b - A_{-i} x_{-i})] + \lambda x_i^2 + \lambda x_{-i}^T x_{-i}$$

Let $\vec{g} = \vec{b} - A_{-i} x_{-i}$

$$f(x) = [(x_i a_i)^T - (g)^T][(x_i a_i) - (g)] + \lambda x_i^2 + \lambda x_{-i}^T x_{-i}$$

$$= \|a_i\|^2 (x_i)^2 - 2(x_i a_i)^T g + \lambda x_i^2 + \lambda x_{-i}^T x_{-i}$$

$$= (\|a_i\|^2 + \lambda) x_i^2 - (2 a_i^T g) x_i + \lambda x_{-i}^T x_{-i}$$

By FONC

$$\frac{\partial f}{\partial x_i} = 2(\|a_i\|^2 + \lambda) x_i^* - 2(a_i^T g) = 0$$

$$x_i^* = \frac{a_i^T g}{\|a_i\| + \lambda}, \text{ where } \vec{g} = \vec{b} - A_{-i} x_{-i}$$

$$x_i^{(t+1)} = \begin{cases} \dfrac{a_i^T (\vec{b} - A_{-i} x_{-i})}{\|a_i\| + \lambda} & \text{if } i = t \bmod d \\ x_i^{(t)}, & \text{else} \end{cases}$$

In the following questions, consider $A \in \mathbb{R}^{m \times d}$ and $\mathbf{b} \in \mathbb{R}^m$ where we pick $d = 100$ and $m = 10$.

2. Write the **pseudocode** for the coordinate descent algorithm to solve the Ridge Regression problem. The pseudocode should describe the steps of the iterative coordinate descent algorithm.

```
Solve_Coordinate_descent(A, b, lambda, x_init, T):
    x ← x_init
    d ← x.length
    t ← 0        // counter

    while t < T:
        i ← t mod d
        x[i] ← a_i^T(b - A_{-i} x_{-i}) / (||a_i|| + lambda)
        increment t by 1
```

```python
import numpy as np
import copy
import matplotlib.pyplot as plt

d = 100
m = 10
A = np.random.normal(0, 1, (m,d))
b = np.random.normal(0, 1, m)
initial_x = np.zeros(d)

def f(x,lambdas):
    error = A@x-b
    error_l2_norm = np.linalg.norm(error)**2
    regularizer = lambdas * np.linalg.norm(x)**2
    return error_l2_norm + regularizer


def coordinate_opt(A,b,lambdas,x,t):
   x_updated = copy.deepcopy(x)
   A_i_th_row = A[:,t] ## select the i-th column
   residual = b - A @ x_updated + A_i_th_row * x_updated[t]
   numerator = A_i_th_row@residual ## above
   denominator = A_i_th_row@A_i_th_row + lambdas ## below
   x_updated[t] = numerator/denominator
   return x_updated

def ridge_regresison_solver(A,b,lambdas,x_init,T=2000):
    x = copy.deepcopy(x_init)
    counter = 0
    ans = []
    while (counter < T):
        t = counter % d
        x = coordinate_opt(A,b,lambdas,x,t)
        ans.append(f(x,lambdas))
        counter += 1
    return ans

ans1 = ridge_regresison_solver(A,b,0.1,initial_x)
ans2 = ridge_regresison_solver(A,b,1.0,initial_x)
ans3 = ridge_regresison_solver(A,b,10,initial_x)
ans4 = ridge_regresison_solver(A,b,100,initial_x)

plt.plot(ans1, label='lambda=0.1')
plt.plot(ans2, label='lambda=1.0')
plt.plot(ans3, label='lambda=10')
plt.plot(ans4, label='lambda=100')

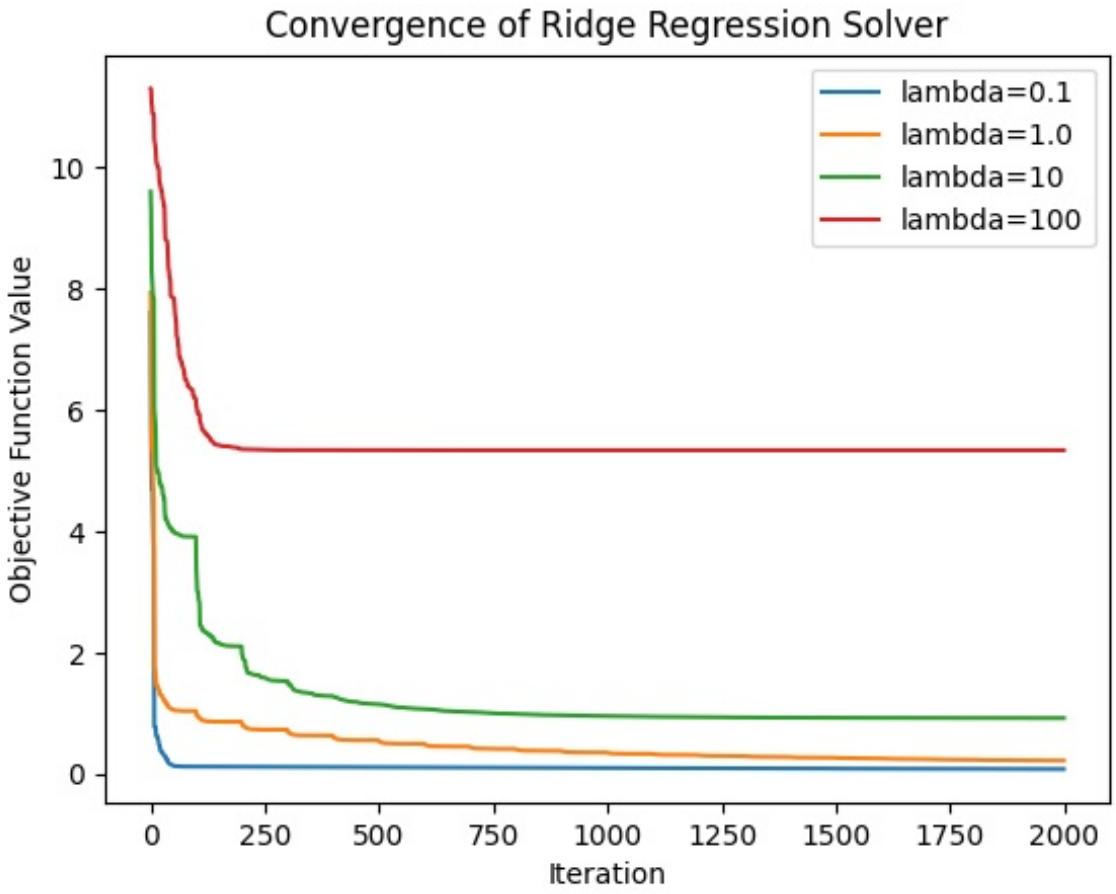
plt.xlabel('Iteration')
plt.ylabel('Objective Function Value')
plt.title('Convergence of Ridge Regression Solver')
plt.legend()
plt.savefig('ridge_regression_convergence.png')
```

→ Problem 3

problem 4

problem 5

Problem 5


Convergence of Ridge Regression Solver
lambda=0.1
lambda=1.0
lambda=10
lambda=100
Objective Function Value
0
2
4
6
8
10
0
250
500
750
1000
1250
1500
1750
2000
Iteration